

EXPLORATORY DATA ANALYSIS

MASOUD MASOUMI

ME 371 - Fall 2025

Department of Mechanical Engineering
The Cooper Union for the Advancement of Science and Art

What is Exploratory Data Analysis?



When given a dataset, the first step is to get to know it, understand its features, and familiarize yourself with the fields.

- **Explore:** Understand the structure, patterns, and characteristics of the data
- **Clean:** Identify and handle missing or incorrect values
- **Transform:** Convert data into a format suitable for analysis
- **Visualize:** Create plots to discover relationships and patterns
- **Summarize:** Extract key statistics and insights

Goal: Maximize insight into a dataset, uncover underlying structure, extract important variables, detect outliers and anomalies



The typical workflow for exploratory data analysis:

1. Import and Inspect Data

- Load the dataset and examine its basic structure
- Check dimensions, column names, and data types

2. Data Cleaning & Wrangling

- Handle missing values (drop or replace)
- Correct data formats and types
- Remove or fix outliers and inconsistencies

3. Descriptive Statistics & Visualization

- Calculate summary statistics
- Create visualizations to understand distributions and relationships

4. Advanced Analysis

- Grouping and aggregation
- Correlation analysis

Our Dataset: EPA Fuel Economy



Dataset: Fuel Economy Data from the U.S. Department of Energy

- Produced by the Office of Energy Efficiency and Renewable Energy
- Vehicle testing done at EPA's National Vehicle and Fuel Emissions Laboratory
- Contains information about:
 - Vehicle specifications (make, model, class, engine)
 - Fuel economy (city MPG, highway MPG)
 - Environmental impact (CO₂ emissions, greenhouse gas score)
 - Transmission type and drivetrain
- Source: <https://www.fueleconomy.gov>

Importing Data with Pandas



First step: Import the necessary libraries and load the data

```
1 import pandas as pd
2
3 xlsx = pd.ExcelFile('URL_to_data.xlsx')
4 df = pd.read_excel(xlsx, 'Sheet1')
5
6 df.head() # Display first few rows
7
8 df.shape # Check dimensions
9
```

Key Pandas methods for initial inspection:

- .head() - View first 5 rows
- .shape - Get dimensions (rows, columns)
- .columns - List all column names
- .dtypes - Check data types of each column

Renaming Columns



Renaming columns improves readability and makes code cleaner

```
1 df.rename(columns={'Veh Class': 'Class'},
2            inplace=True) # Rename single column
3
4 df.rename(columns={
5     'Underhood ID': 'ID',
6     'Stnd Description': 'Description',
7     'Cert Region': 'Region'
8 }, inplace=True) # Rename multiple columns
9
```

Tips:

- Use descriptive but concise names

- Remove special characters and spaces if possible

- Use `inplace=True` to modify the original dataframe. Otherwise, assign to a new variable: `df_new = df.rename(...)`

Data Cleaning: Why It Matters



Real-world data is messy! Common issues include:

Missing Values

NaN, None, empty cells

Can arise from data collection errors, equipment failures, or inapplicable fields

Incorrect Data Types

Numbers stored as strings (e.g., "21/49" instead of 21)

Dates in wrong format

Outliers and Anomalies

Values that don't make physical sense

Data entry errors

Inconsistent Formatting

Mixed units, capitalization, or spelling

Remember: Garbage in, garbage out! Clean data is essential for reliable analysis.

Identifying Missing Values



Pandas provides powerful methods to detect missing data:

```
1 df.isnull() # Check for null values (returns boolean DataFrame)
2
3 df['Fuel'].isnull().value_counts() # Count missing values in a
   specific column
4
5 df[df.isnull().any(axis=1)] # Find rows with any missing values
6
7 df.isnull().any(axis=0) # Find columns with any missing values
8
```

Useful methods:

- .isnull() - Detects NaN, None, NaT
- .notnull() - Opposite of isnull()
- .value_counts() - Counts unique values

Finding Missing Values Across Dataset



Loop through all columns to count missing values:

```
1 missing_data = df.isnull()
2
3 for column in missing_data.columns:
4     print(column)
5     print(missing_data[column].value_counts())
6     print(" ")
7
```

This gives a comprehensive view of data completeness across the entire dataset. **Quick check for total missing values:**

```
1 df.isnull().sum().sum()
2
```

Strategies for Handling Missing Data



Two main approaches: **Drop** or **Replace**

Dropping Data

- Drop entire rows with missing values
- Drop entire columns if mostly empty
- Use when:
 - Small percentage of data affected
 - Missing data is random
 - Column is not useful

Replacing Data

- Replace with mean/median
- *Replace with mode (categorical)
- *Forward/backward fill
- *Interpolation
- Domain knowledge guess
- Use when:
 - Large amount of missing data
 - Pattern in missingness

Important: Document your decisions. Missing data handling impacts results.

Dropping Missing Values



Use `.dropna()` to remove rows or columns with missing values:

```
1 # Drop rows where ALL values are NaN
2 df.dropna(axis=0, how='all', inplace=True)
3
4 # Drop rows where ANY value is NaN
5 df.dropna(axis=0, how='any', inplace=True)
6
7 # Drop rows with NaN in specific columns
8 df.dropna(axis=0, how='all', subset=['Displ', 'Cyl'], inplace=
   True)
9
10 # Drop columns with any missing values
11 df.dropna(axis=1, how='any', inplace=True)
12
13 df.drop(293, axis=0, inplace=True) # Drop specific row by index
14
```

Parameters: `axis` (0=rows, 1=columns), `how` ('any' or 'all'), `subset` (specific columns)

Replacing Missing Values



Use `.replace()` or `.fillna()` to substitute missing values:

```
1 import numpy as np
2
3 # Replace NaN with a specific value
4 df['Fuel'].replace(np.nan, 'Gasoline', inplace=True)
5
6 # Fill with mean (numerical columns)
7 df['Displ'].fillna(df['Displ'].mean(), inplace=True)
8
9 # Fill with median
10 df['Cyl'].fillna(df['Cyl'].median(), inplace=True)
11
12 # Fill with mode (categorical)
13 df['Region'].fillna(df['Region'].mode()[0], inplace=True)
14
15 # Forward fill (ffill) or backward fill (bfill)
16 df['column'].fillna(method='ffill', inplace=True) # OR df['
           column'].ffill(inplace=True)
17
```

Checking Data Types



Data type determines what operations you can perform. Check with:

```
1 # Check data types for all columns
2 df.dtypes
3
4 # Check data type for specific column
5 df['City MPG'].dtype
6
```

Common data types in Pandas:

int64 - Integers

float64 - Floating point numbers

object - Strings or mixed types

category - Categorical variables (efficient for repeated values)

datetime64 - Date and time

bool - Boolean (True/False)

Converting Data Types



Use `.astype()` to convert between types:

```
1 # Convert to numeric types
2 df['Displ'] = df['Displ'].astype('float64')
3 df['Cyl'] = df['Cyl'].astype('int64')
4 df['City MPG'] = df['City MPG'].astype('int64')
5
6 # Convert to categorical (efficient for text)
7 df['Class'] = df['Class'].astype('category')
8 df['SmartWay'] = df['SmartWay'].astype('category')
9 df['Trans'] = df['Trans'].astype('category')
10 df['Drive'] = df['Drive'].astype('category')
11 df['Fuel'] = df['Fuel'].astype('category')
12
```

Why categorical type?

- Saves memory for columns with repeated values

- Enables certain operations (e.g., ordering)

- More efficient for grouping and aggregation

EPA Dataset: Data Type Issues



Problem encountered: Some MPG values stored as strings!

Hybrid Vehicles

Fuel types: "Gasoline/Electricity" and "Ethanol/Gas"

MPG values stored as "21/49" (two modes of operation)

Cannot convert string to integer directly

Solution Options

Split column into two columns (complex)

```
1 df[['City_Mode1', 'City_Mode2']] = df['City MPG'].str.  
    split('/', expand=True)
```

Remove hybrid vehicles from dataset (simpler)

Character Issues

Found "-" character in **Greenhouse Gas Score**

Replaced with appropriate numeric value based on similar vehicles

Lesson: Always check for non-numeric characters before type conversion.

Resetting the Index



After dropping rows, indices become non-sequential. Reset them:

```
1 # Reset index but keep old index as a column
2 df.reset_index()
3
4 # Reset index and drop the old index
5 df.reset_index(drop=True, inplace=True)
6
```

Why reset the index?

- Cleaner looking dataframe

- Prevents confusion with row numbers

- Easier to iterate through rows

- Some operations expect sequential indices

Tip: Always reset index after major data cleaning operations.

Exporting Cleaned Data



After cleaning, you may want to export the data:

```
1 # Export to CSV
2 df.to_csv('EPA_2020_Fuel_Economy.csv')
3
4 # Export to Excel
5 df.to_excel('EPA_2020_Fuel_Economy.xlsx',
6             sheet_name='FirstSheet')
7
8
```

Why export?

- Save cleaned data for future use

- Share with collaborators

- Use in other tools/software

- Create backups at different processing stages

Descriptive Statistics: The Basics



Get quick statistical summary with `.describe()`:

```
1 # Summary statistics for numerical columns
2 df.describe()
3
4 # Include categorical columns
5 df.describe(include='category')
6
7 # Include all columns
8 df.describe(include='all')
9
```

Five-number summary for numerical data:

- Minimum value

- First quartile (25th percentile)

- Median (50th percentile)

- Third quartile (75th percentile)

- Maximum value

Plus: count, mean, and standard deviation



Understand distribution of categorical variables:

```
1 # Count occurrences of each category
2 df['SmartWay'].value_counts()
3
4 # Convert to DataFrame
5 df_Fuel = df['Fuel'].value_counts().to_frame()
6
7 df['Displ'].nunique() # Count unique values
8
9 df['Fuel'].unique() # Get unique values
10
```

Use cases:

- Understand class imbalance

- Identify rare categories

- Check for data entry errors

- Plan for grouping or aggregation

Visualization: Why It Matters



Humans are visual creatures. Visualizations help us:

Identify Patterns

- Spot trends, clusters, and relationships
- See what summary statistics might miss

Detect Anomalies

- Find outliers and unusual data points
- Identify data quality issues

Communicate Results

- Share insights with non-technical audiences
- Make data-driven arguments compelling

Test Hypotheses

- Quickly check if relationships exist
- Guide further statistical analysis

Python Visualization Libraries



Two main libraries for data visualization:

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
```

Matplotlib

- Foundation of Python plotting
- Highly customizable
- More control, more code
- Good for basic plots

Seaborn

- Built on matplotlib
- Beautiful default styles
- Less code, prettier plots
- Great for statistical plots
- Works well with Pandas

Scatter Plots



Explore relationships between two numerical variables:

```
1 # Using Pandas plot method
2 df.plot(kind='scatter', x='Displ', y='Greenhouse Gas Score',
3         figsize=(10,6), s=70, c='k')
4
5 # Using Seaborn with regression line
6 sns.regplot(data=df, x='City MPG', y='Greenhouse Gas Score')
7
8 # Without regression line
9 sns.regplot(data=df, x='City MPG', y='Greenhouse Gas Score',
10            fit_reg=False)
```

What to look for: Linear relationships, clusters, outliers, heteroscedasticity

Pair Plots



Visualize all pairwise relationships at once:

```
1 # Create scatter plot matrix
2 sns.pairplot(df)
3
4 # Select specific columns
5 columns_of_interest = ['Displ', 'City MPG', 'Hwy MPG', '
    Greenhouse Gas Score']
6 sns.pairplot(df[columns_of_interest])
7
8 # Color by categorical variable
9 sns.pairplot(df, hue='Drive')
10
```

Advantages:

- See all relationships simultaneously
- Identify which variables to investigate further
- Diagonal shows distributions of each variable

Warning: Can be slow with large datasets or many variables!



Compare distributions across categorical groups:

```
1 # Box plot with Seaborn
2 plt.figure(figsize=(8,6))
3 sns.boxplot(data=df, x='Drive', y='Greenhouse Gas Score')
4
5 # Multiple categories
6 sns.boxplot(data=df, x='Class', y='City MPG', hue='Drive')
7 plt.xticks(rotation=45) # rotating x-axis label 45 degrees
8
```

Box plots show:

- Median (line in the box)

- Quartiles (box edges)

- Range (whiskers)

- Outliers (individual points)

Histograms and Distribution Plots



Understand the distribution of a single variable:

```
1 # Histogram using Pandas
2 df['Greenhouse Gas Score'].plot(kind='hist',
3                                   bins=10,
4                                   color='blue')
5 plt.xlabel('Greenhouse Gas Score')
6
7 # Distribution plot with Seaborn
8 sns.displot(data=df, x='Greenhouse Gas Score', bins=10)
9
10 # With KDE (kernel density estimate)
11 sns.displot(data=df, x='City MPG', kde=True)
12
```

What to look for: Shape (normal, skewed, bimodal), outliers, typical values

Grouping Data: Split-Apply-Combine



The `groupby` operation follows three steps:

1. **Split:** Separate data into groups based on criteria
Group by one or more categorical variables
2. **Apply:** Perform operation on each group independently
Aggregation (mean, sum, median, count)
*Transformation (normalize within groups)
*Filtration (select groups meeting criteria)
3. **Combine:** Merge results back into a data structure
Return as `DataFrame` or `Series`

Use case: Find average fuel economy by vehicle class, compare emissions across manufacturers, analyze patterns within subgroups

GroupBy: Basic Usage



Group data and apply aggregation functions:

```
1 # Create subset of data
2 df2 = df[['Trans', 'City MPG',
3           'Hwy MPG', 'Air Pollution Score']]
4
5 # Group by transmission type and compute mean
6 df2.groupby('Trans', as_index=False).mean()
7
8 # Other aggregation functions
9 df2.groupby('Trans').median()
10 df2.groupby('Trans').sum()
11 df2.groupby('Trans').count()
12 df2.groupby('Trans').std()
13
14 # Multiple aggregations at once
15 df2.groupby('Trans').agg(['mean', 'median', 'std'])
16
```

GroupBy: Accessing Groups



Query specific groups from the grouped object:

```
1 # Get a specific group
2 df2.groupby('Trans', as_index=False).get_group('AMS-8')
3 # Equivalent to filtering
4 df2[df2['Trans'] == 'AMS-8']
5
6 # Iterate through groups
7 for name, group in df2.groupby('Trans'):
8     print(f"Group: {name}")
9     print(group.head())
10    print()
11
12 # Count number of items in each group
13 df2.groupby('Trans').size()
14
```

GroupBy: Multiple Variables



Group by combinations of multiple variables:

```
1 # Group by two variables
2 df_gp = df[['Class', 'Trans', 'Greenhouse Gas Score']]
3
4 df_grouped = df_gp.groupby(['Class', 'Trans'],
5                             as_index=False).mean()
6
7 # Create pivot table for better visualization
8 grouped_pivot = df_grouped.pivot(index='Trans', columns='Class')
9
10 # Fill missing values
11 grouped_pivot = grouped_pivot.fillna(0)
12
```

Pivot tables make grouped data easier to read and analyze

Correlation: What Is It?



Correlation: A statistical measure of the relationship between two variables

Positive correlation: As X increases, Y tends to increase

Example: Engine displacement and CO₂ emissions

Negative correlation: As X increases, Y tends to decrease

Example: Engine displacement and greenhouse gas score

No correlation: No systematic relationship

Critical reminder: Correlation does NOT imply causation!

Correlation: There is a relationship between X and Y

Causation: Y is influenced by a change in X

Pearson Correlation Coefficient



Pearson correlation coefficient $r(X, Y)$ measures linear relationship strength

Range: $-1 \leq r \leq 1$

Interpretation:

$r = 1$: Perfect positive linear relationship

$r = -1$: Perfect negative linear relationship

$r = 0$: No linear relationship

$|r| > 0.7$: Strong correlation

$0.3 < |r| < 0.7$: Moderate correlation

$|r| < 0.3$: Weak correlation

Assumptions:

Linear relationship

Both variables are continuous

No significant outliers

Data is approximately normally distributed

Computing Correlation in Pandas



Calculate correlation matrix for all numerical variables:

```
1 # Correlation matrix
2 df.corr()
3 # Correlation between two specific variables
4 df['City MPG'].corr(df['Greenhouse Gas Score'])
5
6 # Visualize correlation matrix with heatmap
7 import seaborn as sns
8 plt.figure(figsize=(10,8))
9 sns.heatmap(df.corr(), annot=True, cmap='coolwarm', center=0)
10
```

Tips:

- Heatmaps make patterns easier to spot

- Look for highly correlated variable pairs

- Diagonal is always 1 (variable with itself)

Statistical Significance: p-values



p-value: Probability that observed correlation occurred by chance

```
1 from scipy import stats
2
3 # Pearson correlation with p-value
4 pearson_coef, p_value = stats.pearsonr(
5     df['Hwy MPG'],
6     df['Greenhouse Gas Score']
7 )
8
9 print(f"Correlation: {pearson_coef:.3f}")
10 print(f"P-value: {p_value}")
11
```

Interpretation of p-value:

$p < 0.001$: 99.9% confident (very strong evidence)

$p < 0.01$: 99% confident (strong evidence)

$p < 0.05$: 95% confident (moderate evidence)

$p > 0.05$: Not statistically significant



Key principles for effective exploratory data analysis:

1. Start simple, then dig deeper

Begin with basic statistics and simple plots

Investigate interesting patterns in detail

2. Document your process

Use Jupyter notebooks to combine code, results, and notes

Explain decisions (why dropped rows, why chose mean over median)

3. Validate assumptions

Check data quality at every step

Look for inconsistencies and anomalies

4. Use domain knowledge

Apply engineering judgment

Question results that don't make physical sense

5. Iterate

EDA is not linear - revisit earlier steps as you learn more

Common Pitfalls to Avoid



Mistakes to watch out for during EDA:

Ignoring missing data

- Always check for NaN values

- Understand why data is missing

Not checking data types

- Numbers stored as strings cause errors

- Categorical variables as objects waste memory

Over-relying on statistics alone

- Always visualize.

Confusing correlation with causation

- Correlation shows association, not cause-and-effect

Removing outliers without investigation

- Outliers might be errors OR important discoveries

Summary: EDA Workflow



Complete EDA process for the EPA dataset:

1. Import data and inspect structure
2. Identify and handle missing values
 - Dropped electric vehicles (NaN in Displ, Cyl)
 - Replaced specific missing values using domain knowledge
3. Check and correct data types
 - Removed hybrid vehicles with string MPG values
 - Converted columns to appropriate types
4. Calculate descriptive statistics
5. Create visualizations (scatter plots, box plots, histograms)
6. Group and aggregate data
7. Compute correlations and test significance

Result: Clean dataset ready for modeling or deeper analysis.

Next Steps After EDA



Once EDA is complete, you're ready for:

Feature Engineering

- Create new variables from existing ones

- Transform variables for better model performance

Statistical Modeling

- Linear regression, logistic regression

- Hypothesis testing

Machine Learning

- Predictive models (random forests, neural networks)

- Classification, clustering

Communication

- Create reports and presentations

- Share insights with stakeholders

Remember: Good EDA is the foundation of all data science work.